CLEAR

Owner guide

Guide 1 of 3 · Owner and admin

Run the app like an owner.

What you own, how the admin panel works, where it is hosted for free, and how your users' data is protected.

Clear Audio Recorder · version 1.0 · 10 November 2026 · prepared for Daxator

01

What you own

Clear Audio Recorder is four pieces. You never run a server of your own: the phone app talks straight to services that have free plans.

The phone app
Records, cleans and stores audio on the phone only.

→

Firebase (free plan)
Google sign-in, the user list, app settings.

→

Your admin panel
A small website only you can open.

Google Play
Sells the $2 subscription and pays you.

+

RevenueCat
Tells the app who is Pro, on any phone.

+

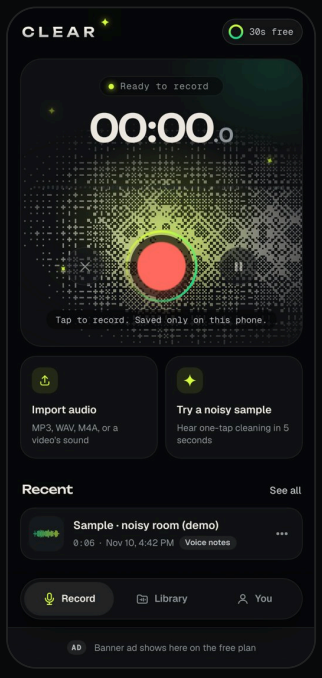
AdMob
Shows ads to free users and pays you.

Why there is no server

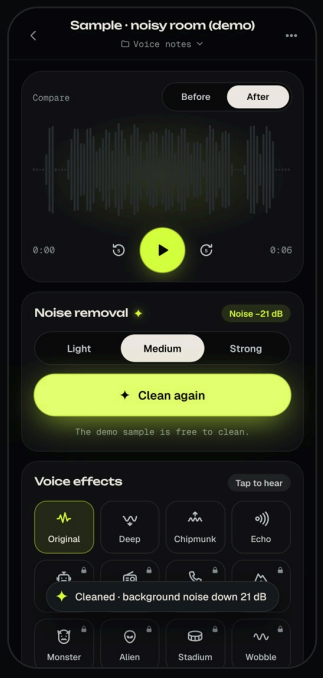
All audio work happens on the user's phone, so there is nothing heavy to host. Sign-in, the database and the rules that guard it are run by Google's Firebase. Its free Spark plan allows 50,000 database reads and 20,000 writes a day, 1 GiB of stored data and 50,000 monthly sign-in users. That covers roughly your first several thousand active users before you would pay anything.

02

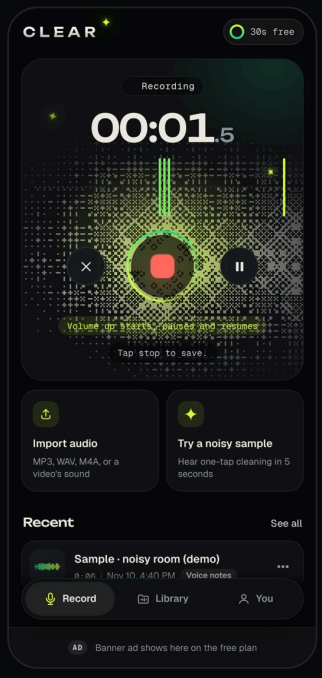
What users can do in version 1.0



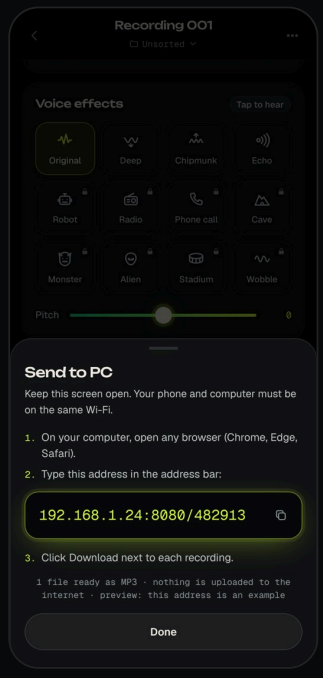
Record screen



One-tap clean, before and after



Recording with the volume button



Send to PC over Wi-Fi

Feature	How it works
Volume-button recording	You → Recording → choose Vol up, Vol down or Either. On the Record screen that button starts, pauses and resumes. The screen stays awake while recording. Phones do not let apps use the power button, so that option is not offered.
Send to PC	Library → Send to PC (or a recording's menu). The phone shows an address like `192.168.1.24:8080/482913`. Type it into any browser on a computer on the same Wi-Fi and click Download. Nothing goes through the internet. The link dies when the sheet is closed or after 15 minutes.
Restore on a new phone	Sign in with Google and the Pro plan, used free seconds, folder names and settings come back. Audio does not: it lives only on the phone it was recorded on, by design.
Delete account	You → Delete account removes the user's profile from your database. Google Play requires this option.

03

Set up Firebase (one time, about 25 minutes)

1. Go to **console.firebase.google.com** and create a project named **Clear Audio Recorder**. Stay on the free Spark plan.

   `Firebase console → Add project`

2. Turn on Google sign-in.

   `Build → Authentication → Get started → Sign-in method → Google → Enable → Save`

3. Create the database in **production mode**, any region near your users.

   `Build → Firestore Database → Create database`

4. Open the file `firebase/firestore.rules` from the project, copy everything, paste it over the rules shown, and press **Publish**. This is the lock on your user data.

   `Firestore Database → Rules`

5. Add a **Web app**. Copy the four values (apiKey, authDomain, projectId, appId) into **both** `src/native/config.js` and `admin/config.js`.

   `Project settings → Your apps → Add app → Web`

6. Copy the **Web client ID** into `googleWebClientId` in `src/native/config.js`.

   `Authentication → Sign-in method → Google → Web SDK configuration`

7. Add an **Android app** with package name `com.daxator.clearaudiorecorder` and paste the **SHA-1** fingerprint from your `SECRETS-TO-ADD.txt` file (Guide 2, step 4). After your first Play upload, add the second SHA-1 shown in Play Console → App integrity.

   `Project settings → Your apps → Add app → Android`

If Google sign-in fails on the phone

Nine times out of ten the SHA-1 fingerprint is missing or belongs to a different key. Use the **signed** APK from GitHub (not the test APK) and check both fingerprints are in Firebase.

04

Host the admin panel for free

The admin panel is the `admin` folder: four small files, no server code. Any static host works. Two that allow commercial use on their free plans and accept a dragged folder:

Cloudflare Pages

dash.cloudflare.com → Workers & Pages → Create → Pages → Upload assets. Drag the `admin` folder. You get an address like `clear-admin.pages.dev`.

Netlify

app.netlify.com → Add new site → Deploy manually. Drag the `admin` folder. You get an address like `clear-admin.netlify.app`.

1. Fill in `admin/config.js` first (step 5 above), then upload the folder.
2. Tell Firebase this website is allowed to sign in: add your new address.
   `Authentication → Settings → Authorized domains → Add domain`
3. Open your admin address and press **Continue with Google**. It will say "This account is not an admin" and show a long ID. Copy it.
4. Make yourself the admin: create a collection named `admins`, with the Document ID set to that long ID, and one field `role` = `owner`. Reload the admin page.
   `Firestore Database → Data → Start collection`

Only people listed in that `admins` collection can open the panel, and only the project owner can edit the collection, from inside the Firebase console. To update the panel later, upload the folder again.

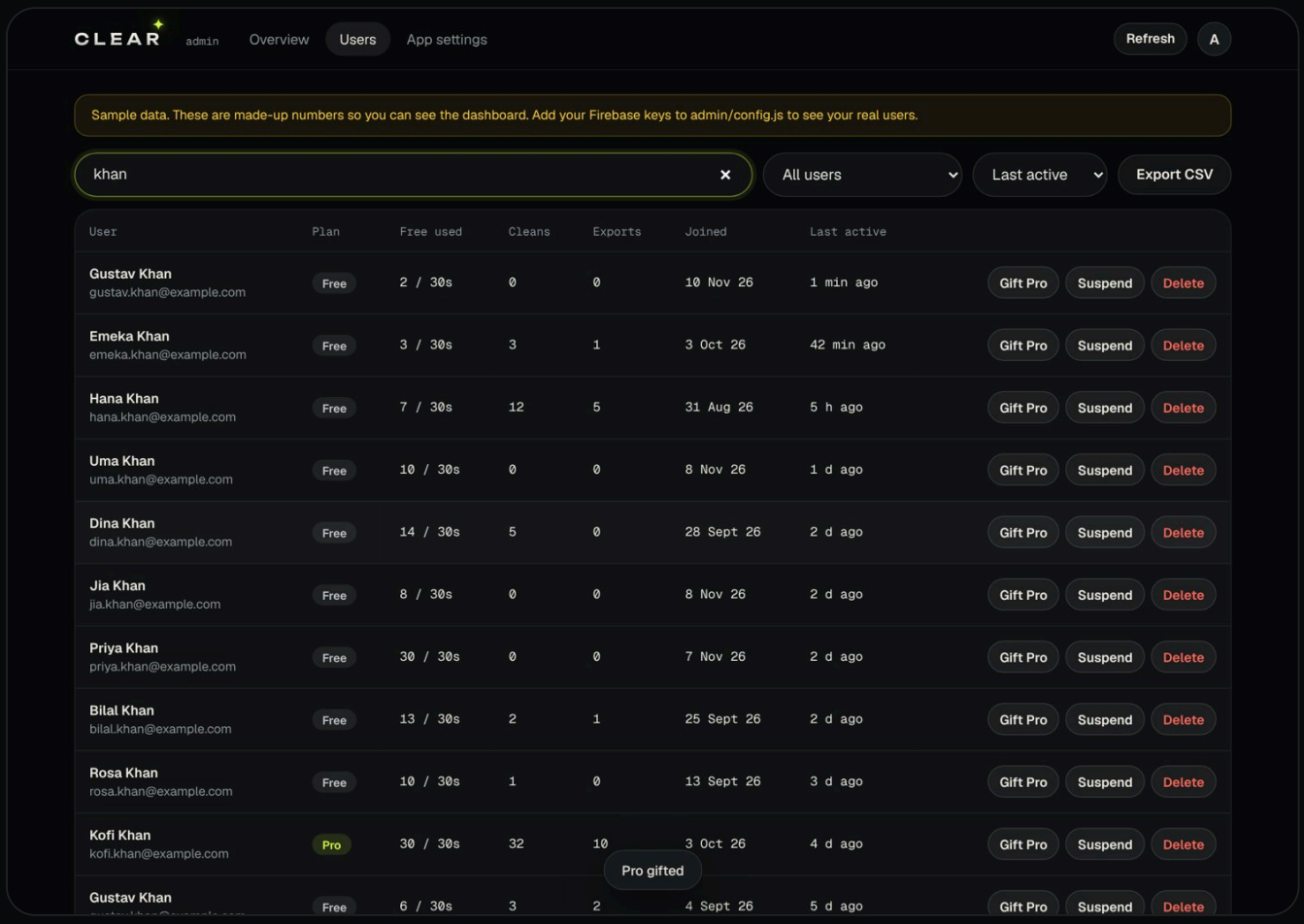
05

Using the admin panel



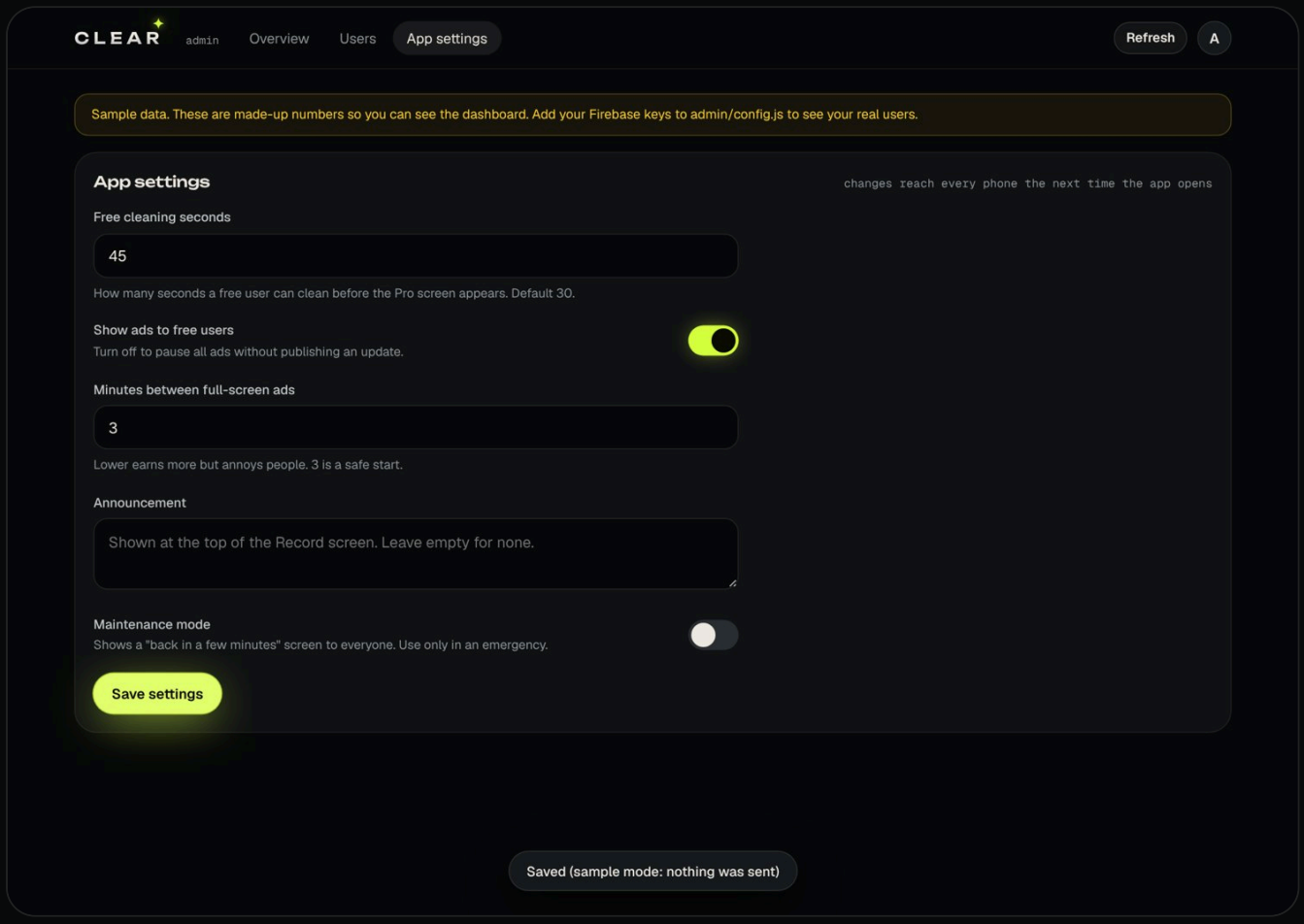
Overview. The numbers in this picture are sample data; yours appear once Firebase is connected.

On the Overview	What it tells you
Estimated monthly revenue	Pro subscribers × $2, minus the store's 15%. It is an estimate from what phones report. The exact figure is in Play Console and RevenueCat.
Signed-in users, active in 7 days	How many people have accounts and how many came back this week. People who never sign in are not counted here; Play Console shows total installs.
New sign-ups, daily active users	30-day bar charts. Watch these after every marketing push.
What people do	Cleans, recordings and exports per day.
Free to Pro funnel	Signed in → cleaned once → used all free seconds → subscribed. The biggest drop shows where to work next.
Plans, app versions, most active users	Paying vs gifted vs free, who is on an old version, and your power users.



Users. Search, filter, gift Pro, suspend, delete, export CSV.

Button	What it does	When to use it
Gift Pro	Gives that account every Pro feature without charging.	Reviewers, influencers, your testers, apologising for a bug.
Suspend	Signs the account out and blocks sign-in.	Abuse only. It does not stop a paid subscription; refund in Play Console.
Delete	Erases the profile from your database.	When someone emails asking for their data to be deleted.
Export CSV	Downloads the filtered list.	Monthly backup. Treat the file as private: it has emails.



App settings. Changes reach every phone the next time the app is opened.

Setting	Effect
Free cleaning seconds	How much a free user can clean before the Pro screen. Start at 30. Raise it if few people reach the paywall, lower it if many reach it but few pay.
Show ads / minutes between full-screen ads	Pause ads instantly, or change how often the full-screen ad shows after an export.
Announcement	One line at the top of the Record screen: a sale, a new feature, a known problem.
Maintenance mode	Blocks the app for everyone with a "back in a few minutes" screen. Emergencies only.

Your routine

Daily, 2 minutes: open Overview, check sign-ups, active users and revenue. **Weekly:** read the funnel, reply to Play Store reviews, export the CSV. **Monthly:** compare the estimate with the real payout in Play Console and AdMob, and adjust free seconds or ad timing once, then wait two weeks before changing again.

06

Security: what protects your users

Audio never leaves the phone

There is no audio on any server to leak. Phone backups are switched off for the app, so recordings do not end up in a cloud backup either.

Database rules

A user can read and change only their own profile. Nobody but an admin can list users. A user cannot make themselves Pro, admin, un-suspended, or lower their used free seconds.

Admin list lives in Firebase

The admin panel's web address is not a secret and does not need to be. Without being in the `admins` list, the database refuses every request.

No secret keys in the app

Everything inside the app and admin files is a public ID. Payments are verified by Google and RevenueCat, not by the app.

Send to PC

Works only on the local network, behind a random 6-digit code, serves only the files chosen, locks after 20 wrong codes and switches off after 15 minutes.

Admin website

Sent with headers that stop it being framed or indexed by search engines, and a content policy that only allows Google sign-in scripts.

What nobody can promise

No app is impossible to hack. What this design does is keep the valuable target small: the only personal data you hold is names, emails and usage counts, and it sits behind Google's sign-in and your rules. A determined person with a rooted phone can still unlock Pro features on their own phone; that costs you one $2 sale, not your user base. The database rules are written and reviewed, but they have not been run against a live Firebase project yet. Do the 3-minute check below before launch.

Your eight security habits

- [ ] Turn on **2-step verification** on the Google account that owns Firebase, Play Console and AdMob. This matters more than anything else on this page.
- [ ] Use a separate Google account for the business, not your everyday one.
- [ ] Never paste a **service account** file or any "private key" into the app, the admin folder or a chat.

- [ ] Keep the GitHub repository **private** and keep `upload-key.jks` out of it.
- [ ] Do not change the Firestore rules to "test mode". If Firebase emails you that your rules are open, fix it that day.
- [ ] Add a second admin only by adding their ID in the `admins` collection; remove it when they leave.
- [ ] In Google Cloud console → Credentials, restrict the Android API key to your package name and SHA-1.
- [ ] Later, turn on **Firebase App Check** (Play Integrity) so only your real app can talk to the database.

3-minute rules check before launch

In Firebase → Firestore → Rules → **Rules Playground**: (1) simulate a **get** on `/users/abc` while signed in as a different user: it must be denied. (2) Simulate a **list** on `/users` as a normal user: denied. (3) Simulate an **update** on your own profile that sets `giftPro` to true: denied. If all three are denied, the lock works.

Sources: Firebase pricing (firebase.google.com/pricing), checked 10 November 2026.